S2 Text.

R-Aevol build, configuration and the yoked-control derivation

Supporting information for "Environmental fluctuation strengthens gene regulation only when it can be sensed"

1. Software and build

We used the regulation-enabled build of Aevol (R-Aevol) from the `aevol-7` branch of https://gitlab.inria.fr/aevol/aevol.git, commit `33277374`. The default branch (`aevol-9`) does not include regulation. Regulation is a compile-time option that defines `__REGUL`; its sources are in `src/libaevol/raevol/`. The build used CMake with

```
cmake .. -Dwith-raevol=ON -Dwithout-stats=OFF -Dwith-x=OFF \
         -DCMAKE_BUILD_TYPE=Release -DCMAKE_POLICY_VERSION_MINIMUM=3.5 \
         -DCMAKE_CXX_FLAGS="-Xpreprocessor -fopenmp -I/opt/homebrew/opt/libomp/include" \
         -DCMAKE_EXE_LINKER_FLAGS="-L/opt/homebrew/opt/libomp/lib -lomp"
```

`-Dwithout-stats=OFF` is required for the per-generation statistics file used in all analyses; the option is on by default and suppresses it. Building on Apple Silicon (macOS, Apple clang 21, Homebrew `cmake`, `libomp` and `boost`) needed changes that do not affect the model. The policy flag above is required because the bundled GoogleTest declares a CMake minimum below 3.5. The OpenMP include directory had to be passed through `CMAKE_CXX_FLAGS`, since it did not reach every target otherwise. Line 118 of `CMakeLists.txt` hard-codes the Intel Homebrew path `/usr/local/lib/libomp.dylib`, which we changed to `/opt/homebrew/opt/libomp/lib/libomp.dylib`. All simulations ran single-threaded, ten or twelve at a time, on one 12-core machine.

2. Parameter file

Each run was created with `aevol_create` from a parameter file generated by `replication/raevol/gen_params_v2.py`. The exploratory runs used `gen_params_v1.py` and the grain experiment `gen_params_grain.py`, which imports the same template. The template, with the fields that vary between runs shown in braces, is:

```
STRAIN_NAME             {name}
SEED                    {seed}
INIT_POP_SIZE           {1024 or 512}
WORLD_SIZE              32 {32 or 16}
INIT_METHOD             ONE_GOOD_GENE CLONE
CHROMOSOME_INITIAL_LENGTH   5000
FUZZY_FLAVOR            3
SELECTION_SCHEME        fitness_proportionate   750
POINT_MUTATION_RATE     5e-5
SMALL_INSERTION_RATE    5e-5
SMALL_DELETION_RATE     5e-5
MAX_INDEL_SIZE          6
WITH_ALIGNMENTS         false
DUPLICATION_RATE        5e-5
DELETION_RATE           5e-5
TRANSLOCATION_RATE      5e-5
INVERSION_RATE          5e-5
BACKUP_STEP             10000
RECORD_TREE             false
HILL_SHAPE_N                    4
HILL_SHAPE_THETA                0.5
DEGRADATION_RATE                1
NB_DEGRADATION_STEP             10
NB_INDIV_AGE                    20
```

```
INDIVIDUAL_EVALUATION_AGES     1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
BINDING_ZEROS_PERCENTAGE       0.75
RANDOM_BINDING_MATRIX          false
WITH_HEREDITY                  false
PROTEIN_PRESENCE_LIMIT         1e-2
ENV_SAMPLING            300
MAX_TRIANGLE_WIDTH      0.033333333
ENV_AXIS_FEATURES       METABOLISM
ENV_VARIATION           switch_in_a_list {q, or 1.5q for yoked arms}
NB_ENVIRONMENTS {2 or 4}
```

followed by the target and signal definitions of Section 3. The exploratory runs used `BACKUP_STEP` and `TREE_STEP` 10,000 as well, and an identical template otherwise. With `RANDOM_BINDING_MATRIX false` the platform reads `binding_matrix.rae` from the run directory; every run used the file distributed with `examples/dynamic_env`, unchanged. The mutation and rearrangement rates are those of `examples/complex_env`, ten times those of `examples/dynamic_env`.

3. Targets and signals

Each environment is a target function defined by three Gaussians (`ENV_ADD_GAUSSIAN` *env height mean width*):

	Gaussian 1	Gaussian 2	Gaussian 3
Target A (height, mean, width)	1.2, 0.52, 0.12	−1.4, 0.50, 0.07	0.3, 0.80, 0.03
Target B (height, mean, width)	1.2, 0.48, 0.12	−1.4, 0.50, 0.07	0.3, 0.20, 0.03

B is A reflected about 0.5. At creation the platform reports a geometric area of 0.152468 for each. Frozen arms define two copies of A (odd seeds) or of B (even seeds). The two signal proteins are taken from `examples/complex_env` (its first two `CREATE_SIGNAL` lines):

```
CREATE_SIGNAL h0 w0 h1 m1 w0 h1 m0 h0 h1 w0 h0 m1 h1 h1 m1 m0 h0 m1 w0 h1 m1 w0 h1 m0 h0 h1 w0 h0 m1 h1 h1 m1
    m0 h1 m0 m1
CREATE_SIGNAL m0 h0 m1 h1 m1 w0 m0 m1 m0 h0 m1 h1 w0 h0 h0 h1 m1 m0 h1 w0 h1 h0 m1 h1 m0 w0 w0 m0 m1 w0 w0
    h1 h0 w0 h1 h0 h0 m0 h0 w0 h0 m1 m0 w0 h1 w0 w0 h1 m0
```

They are attached to environments with `ENV_ADD_SIGNAL` *env signal*:

Arm	Environments	`ENV_ADD_SIGNAL` lines
Informative	(A), (B)	1 1; 2 2
Yoked	(A), (A), (B), (B)	1 1; 2 2; 3 1; 4 2
No cue	(A), (B)	none

Before the first batch was launched, the platform's creation log was checked for each type of arm and reported the expected signal for each environment.

4. How environments switch

The simulations run on the platform's SIMD engine. Its environment handler (`SIMD_PhenotypicTargetHandler_R::ApplyVariation`) builds, each generation, the sequence of environments for the 20 ages of a lifetime, starting from the environment of the last age of the previous generation:

```
id_new_env = list_of_old_target_id[nb_indiv_age_ - 1];
id_old_env = list_of_old_target_id[nb_indiv_age_ - 1];
```

```
for (int16_t i = 0; i < nb_indiv_age_ ; i++) {
  double env_chang = var_prng_->random();
  if (env_chang < env_switch_probability_) {
    while (id_new_env == id_old_env)
      id_new_env = var_prng_->random(nb_env_);
    id_old_env = id_new_env;
  }
  list_env_id_[i] = id_new_env;
  ...
}
```

The environment is therefore a single Markov chain over ages that continues across generations. At each age it switches with probability q, and a switch moves to one of the *other* environments chosen uniformly by rejection sampling. The sequence is shared by the whole population. The non-SIMD handler contains a branch that, when a `deterministic` flag is set and there are two environments, gives each lifetime a fixed half-and-half split; the flag is initialised to `false` and never set, and `check_simd`, which would make the SIMD engine copy the non-SIMD sequence, is never enabled by `aevol_run`. The branch is therefore never executed.

During development, at each age the concentrations of all signal proteins are set to zero and those attached to the environment active at that age are set to 0.9, before the regulatory dynamics for that age are integrated. We checked this in the SIMD engine's network solver (`src/libaevol/7/ExpManager_7.cpp`), which is the code path used, as well as in the non-SIMD implementation (`Individual_R::EvaluateInContext`).

5. Matching switching rates in the yoked arm

Let an environment be a pair (target, signal). In the informative arm the two environments are (A, s1) and (B, s2), so every switch changes the target and the signal together. With switching probability q per age, the target changes at rate q per age and the signal at rate q per age.

In the yoked arm there are four environments, (A, s1), (A, s2), (B, s1) and (B, s2). From any of them, a switch moves to one of the three others with equal probability. Of the three, one keeps the target and changes the signal, one changes the target and keeps the signal, and one changes the two together. A switch therefore changes the target with probability 2/3 and the signal with probability 2/3. With switching probability q', each changes at rate $2q'/3$ per age, and setting $q' = 1.5q$ gives each a rate of q, as in the informative arm. Without this correction the yoked arm would fluctuate a third less often than the informative arm, and any difference between the two would be confounded with the rate of fluctuation.

The two arms also match in the marginal frequency of each signal (one half) and of each target (one half), and in the number of signal proteins present at any age (one). They differ in the association between signal and target. In the informative arm the signal determines the target, and in the yoked arm the two are independent. They also differ in the joint dynamics of a switch, since in the informative arm target and signal always change together and in the yoked arm they do so in one switch out of three. This is a direct consequence of removing the association and cannot be matched as well.

6. Measures

R-Aevol writes `stats/stats_simd_best.csv`, one row per generation, for the individual with the highest fitness. The regulatory statistics are computed in `src/libaevol/7/Stats_7.cpp`:

```
for (auto affinity: rna->affinity_list) {       // coding RNAs only
  if (affinity.enhancer_factor > 0.0) { nb_activators++; mean_activator_activity += affinity.enhancer_factor
     ; }
  if (affinity.operator_factor > 0.0) { nb_operators++;  mean_operator_activity  += affinity.operator_factor
     ; }
}
```

```
nb_influences_       = nb_operating_influences_ + nb_enhancing_influences_;
av_value_influences_ = (mean_activator_activity + mean_operator_activity) / (nb_activators + nb_operators);
```

The column `nb_link` is the number of positive enhancer factors plus the number of positive operator factors over all protein–RNA affinities (an affinity with a positive factor of each kind is counted twice), and `avg_influence` is the mean of those factors, so the total regulatory strength is $S = \texttt{nb_link} \times \texttt{avg_influence}$. When there are no links, `avg_influence` is undefined and we set $S = 0$ (an earlier, exploratory script dropped such generations instead; the difference is immaterial for the data reported, since they are rare). Genome size is `nb_functional_genes`. Each run is summarised by the mean of each measure over generations 25,001–30,000, and the run is the unit of analysis.

7. Runs and seeds

Experiment	Cells	Seeds
Exploratory (72 runs)	6 arms at $q = 0.10$, population 1024	$100000 + 1000\,a + s$, $a = 0..5$, $s = 1..12$
Pre-registered sweep, primary (168)	4 arms at $q = 0.05, 0.10, 0.20$; frozen no-cue and frozen yoked at $q = 0.10$; population 1024	$200000 + 1000\,c + s$
Pre-registered sweep, secondary (48)	fluctuating informative, yoked and no-cue, frozen informative; $q = 0.10$; population 512	$200000 + 1000\,c + s$
Grain (60)	fluctuating no-cue at $q = 0.1, 0.01, 0.001, 0.0002$; frozen no-cue	$300000 + 1000\,c + s$

The indices a and c number the arm or cell in the order written by the generator script and s the replicate. Runs were launched seed-major, so that stopping a batch early would still leave a balanced design. Every run reached 30,000 generations with exit status 0.

8. Pre-registration record

Item	Commit	Content
Sweep P1–P4	`f985ccd`	design, predictions, decision rules; `gen_params_v2.py`, `analyse_v2.py`
EvoSim confirmation P5	`4bc16c3`	viability-pilot rule, analysis script
P5 outcome	`56fe3a0`	no grid met the rule; not run
Grain G1–G4	`fa00777`	`gen_params_grain.py`, `analyse_grain.py`

Before each analysis ran, the SHA-256 checksum of the analysis script on the simulation machine was compared with the committed version (`analyse_v2.py` prefix `d0609140a970de48`; `analyse_grain.py` prefix `89a8613289731999`). Each analysis was run once, on complete data. The simulation machine's idle-sleep setting suspended runs during the first night of the sweep; this lengthened wall time but cannot change results, which are determined by the seed. From that point runs were protected by `caffeinate`. No other deviation occurred.

9. Results of every pre-registered test

Means of the two cells compared, two-sided permutation p on the difference of means (100,000 permutations), the Holm-adjusted p used for the verdicts (across the 12 tests of P1–P4, and across the four grain tests), and the p of the rank-based permutation test computed alongside. The rank-based tests lead to the same verdicts. P1 and P2 at $q = 0.05$ are listed but were not evaluable, because the manipulation

check failed at that rate. Values are copied from `replication/raevol/results_v2_preregistered.txt` and `results_grain_preregistered.txt`.

Test	Cell	Mean A	Mean B	p	Holm p	Rank p
P1	$q = 0.05$	11.03	5.95	0.0046	0.0089	0.0047
P1	$q = 0.10$	13.39	7.53	0.0015	0.0046	0.0027
P1	$q = 0.20$	13.18	4.80	$< 10^{-4}$	0.0001	$< 10^{-4}$
P2	$q = 0.05$	11.03	6.32	0.0044	0.0089	0.0020
P2	$q = 0.10$	13.39	5.73	$< 10^{-4}$	0.0001	$< 10^{-4}$
P2	$q = 0.20$	13.18	4.56	$< 10^{-4}$	0.0001	$< 10^{-4}$
P3	$q = 0.05$	1.11	6.30	$< 10^{-4}$	0.0001	$< 10^{-4}$
P3	$q = 0.10$	1.25	6.30	$< 10^{-4}$	0.0001	$< 10^{-4}$
P3	$q = 0.20$	1.14	6.30	$< 10^{-4}$	0.0001	$< 10^{-4}$
P4	$q = 0.05$	26.28	16.34	$< 10^{-4}$	0.0001	$< 10^{-4}$
P4	$q = 0.10$	28.28	18.07	$< 10^{-4}$	0.0001	$< 10^{-4}$
P4	$q = 0.20$	25.60	18.02	$< 10^{-4}$	0.0003	0.0002
G1	coarse vs fine	19.24	19.45	0.801	1.000	0.885
G2	coarse vs frozen	19.24	25.58	0.0001	0.0004	0.0003
G3	coarse vs fine	2.04	2.29	0.600	1.000	0.756
G4	coarse vs frozen	2.04	8.40	$< 10^{-4}$	$< 10^{-4}$	$< 10^{-4}$

P1: S, fluctuating informative (A) against frozen informative (B). P2: S, fluctuating informative against fluctuating yoked. P3: S, fluctuating no-cue against frozen no-cue. P4: functional genes, frozen informative against fluctuating informative. G1 and G2: functional genes; G3 and G4: S; coarse grain ($q = 0.0002$, A) against fine grain ($q = 0.1$) or the frozen arm (B).